

游时

GAMETIME

桌面游戏时长记录器

产品说明文档

手动添加　自动计时　卸载不丢记录　本地存储

版本 1.1.2 · Windows 10 / 11 x64 · 2026

Python 3.14 · PySide6 / Qt 6.12 · SQLite

产品说明文档

游时 GameTime · 桌面游戏时长记录器 · v1.1.2

项目	内容
产品名称	游时 GameTime
版本	1.1.2
平台	Windows 10 / 11 x64
产品形态	绿色便携版（无需安装）
技术栈	Python 3.14.3 · PySide6 / Qt 6.12 · SQLite · psutil · PyInstaller 6.20
文档日期	2026 年 10 月 20 日

本文档介绍游时的产品定位、功能、使用方法、技术框架、关键实现、测试结果与部署方式，适用于最终用户、验收人员与后续维护者。

阅读指南

- **普通用户：**想快速上手：阅读第 3 章「使用说明」。
- **验收人员：**想了解“为什么不会出现未知游戏”：阅读 2.7 与 5.6。
- **开发 / 维护者：**想了解技术选型与架构：阅读第 4、5 章。
- **测试人员：**测试结果与复现命令：阅读第 6 章，完整明细见 docs/测试报告.md。

目录

01 **产品概览**

02 **功能清单**

03 **使用说明**

04 **框架与技术选型**

05 **关键实现**

06 **测试报告**

07 **部署与限制**

08 **交付与声明**

01 产品概览

游时是一款本地优先的 Windows 桌面游戏助手，专注记录游戏库中的桌面游戏时长。你可以手动选择游戏 exe，也可以扫描 Steam、Epic 与常见 Games 目录，确认候选后再加入。扫描不会静默把结果塞进库，这种边界让数据准确、界面简单，也让卸载后的历史处理更可控。

1.1 设计原则

原则	落地方式
添加可控	手动选择，或扫描后勾选确认；绝不静默加入未确认候选
记录准确	2 秒轮询、进程退出即结束、10 秒心跳、睡眠大间隔最多计 5 秒
卸载不丢记录	添加时永久缓存名称和图标；exe 消失后保留历史，可重新定位
外观可适应	深色 / 浅色 / 跟随系统，切换即时生效
本地优先	SQLite + 本地图标；无账号、无云端、无网络请求
克制、不打扰	关闭窗口进入托盘继续计时；通知默认关闭



图 1 概览：今日游玩、实时计时状态、最近游玩与近 7 天趋势

02 功能清单

2.1 两种添加方式

方式 A：手动添加。在「添加游戏」中选择游戏的 .exe，游时读取版本信息中的产品名作为默认名称，并从 exe 提取图标永久缓存；提取失败时生成品牌色字母封面。同一路径只能添加一次。



图 2 添加游戏：名称、程序路径、备注与图标预览

方式 B：自动扫描后确认。从「游戏库」或「概览」点击「扫描游戏」，读取 Steam libraryfolders/appmanifest、Epic manifest、常见 Games 文件夹、用户添加的自定义目录，并默认全盘扫描所有本地固定磁盘。全盘扫描会跳过系统与缓存目录，按名称、层级、文件大小评分挑选候选，并过滤安装器、运行库、平台启动器和已知工具；结果必须勾选确认后才写入游戏库。

实际例子：自定义安装的《以闪亮之名》位于 D:\APP2\Yslzm\...，快速扫描不会发现；全盘扫描可找到 Azure-Win64-Shipping.exe，并结合注册表名称显示为「以闪亮之名」。全盘扫描默认开启，取消勾选可只做快速扫描。

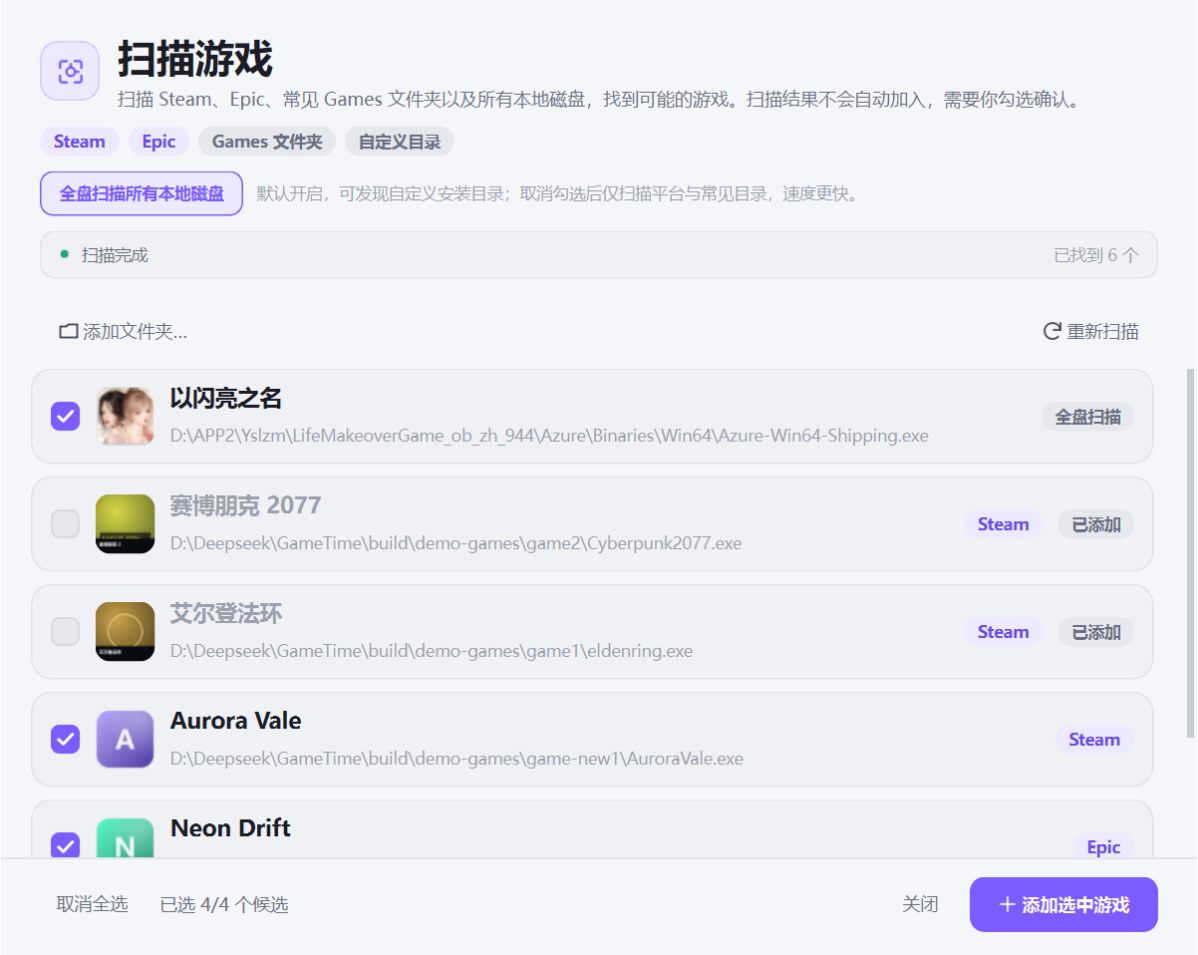


图 3 扫描游戏：来源、候选、已添加状态与确认添加

2.2 双轨计时

模式	触发方式	结束方式	适用场景
自动计时	已添加游戏的 exe 进程出现	进程退出	直接启动 exe 的单机游戏
手动计时	详情页点击「开始手动计时」	用户点击结束	启动器、特殊封装、权限受限程序

自动计时只匹配添加时选择的完整 exe 路径，不做进程名猜测。游戏在添加前已经在运行，也会在下一轮轮询中被发现。手动会话不受进程扫描影响，直到用户主动结束；同一款游戏同时只会存在一个进行中的会话。

2.3 游戏库

卡片展示图标、名称、累计时长、对局数、上次游玩与状态。支持按名称或文件名搜索，按最近游玩、游玩时长、名称排序。绿色「计时中」代表正在记录；「文件缺失」代表 exe 不在了，但历史仍保留。

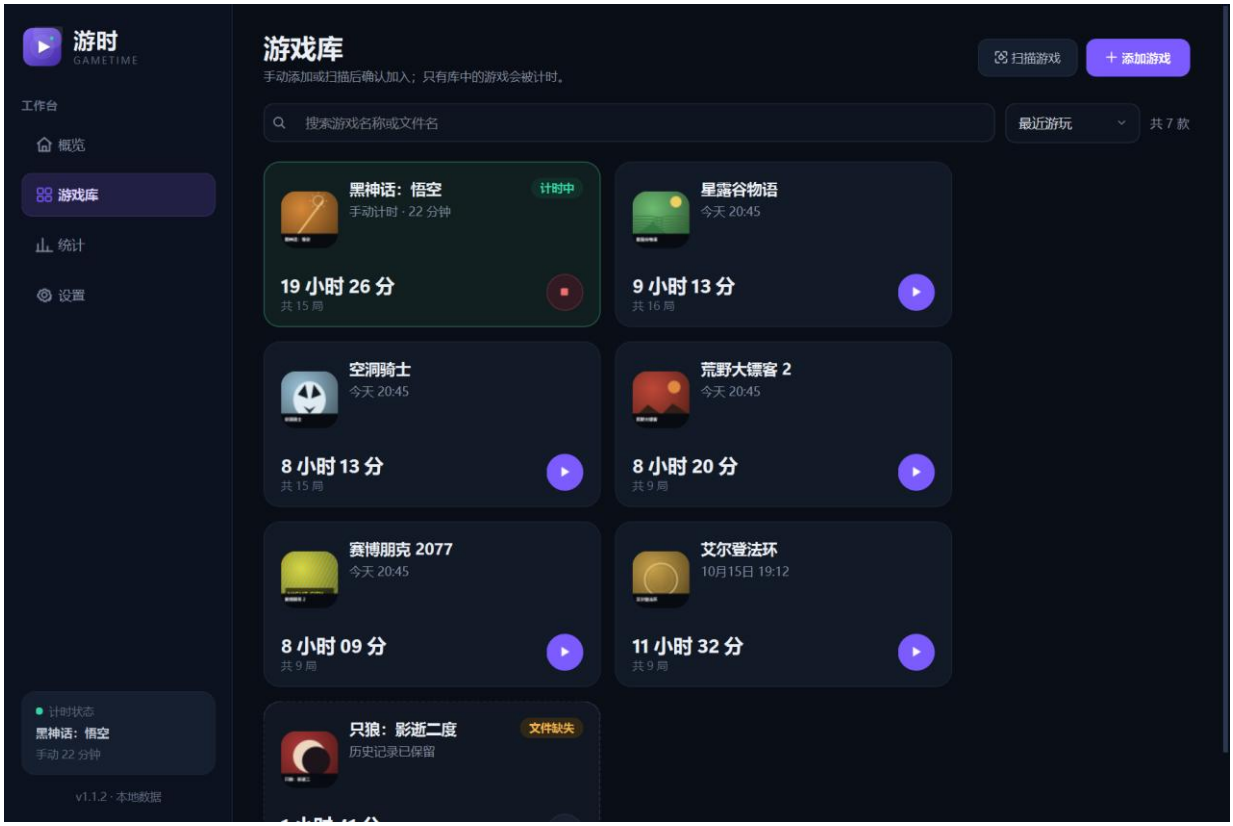


图 4 游戏库：搜索、排序、计时中卡片与文件缺失卡片

2.4 游戏详情与历史

详情页集中展示该游戏的总时长、对局数、近 7 天、上次游玩、近 7 天柱状图与最近会话。同时提供启动、手动计时、编辑、移除与重新定位。关闭详情不会停止计时。



图 5 详情：总时长、周趋势、会话历史与手动计时

2.5 统计

统计页展示今日、近 7 天、累计时长，近 14 天按自然日的柱状图，以及游戏排行 Top 5。跨午夜的会话会按时长比例拆分到两个自然日；电脑睡眠的时间不计入游玩时长。



图 6 统计：今日 / 近 7 天 / 累计、14 天趋势与排行

2.6 主题、托盘与设置

设置页提供深色、浅色、跟随系统三种主题。切换会立即更新 QSS、QPalette、原生标题栏、图表、开关、卡片和图标颜色，并重建主窗口内容；当前页面与计时服务保持不变。跟随系统每 3 秒读取一次 Windows 个性化设置。



图 7 浅色主题与设置：主题选择器、托盘、通知、数据

关闭主窗口默认隐藏到托盘，计时继续；托盘菜单可打开主界面、查看计时状态或退出。设置还提供关闭到托盘、开机自启、启动时最小化、计时通知、打开数据目录与 CSV 导出。开机自启只写当前用户注册表，不需要管理员权限。

2.7 卸载后的行为（重点）

游时从设计上消除了“未知游戏”状态，而不是在界面层简单隐藏一个标签。

- **添加即缓存**：名称、图标、强调色、路径写入 SQLite；图标另存为 PNG。
- **不自动清理**：游戏文件消失后，不删除、不隐藏、不改名该条目。
- **图标不依赖 exe**：优先使用缓存 PNG；缓存也不存在时，用名称生成字母封面，而不是问号。
- **状态准确**：显示「文件缺失」和「历史记录已保留」，这是文件状态，不是未知游戏。
- **历史照常统计**：总时长、对局、排行继续包含该游戏。
- **可恢复**：重新定位到新 exe 后，记录与图标继续使用。
- **由用户决定删除**：只有主动「移除」并确认后，才删除条目及其历史。

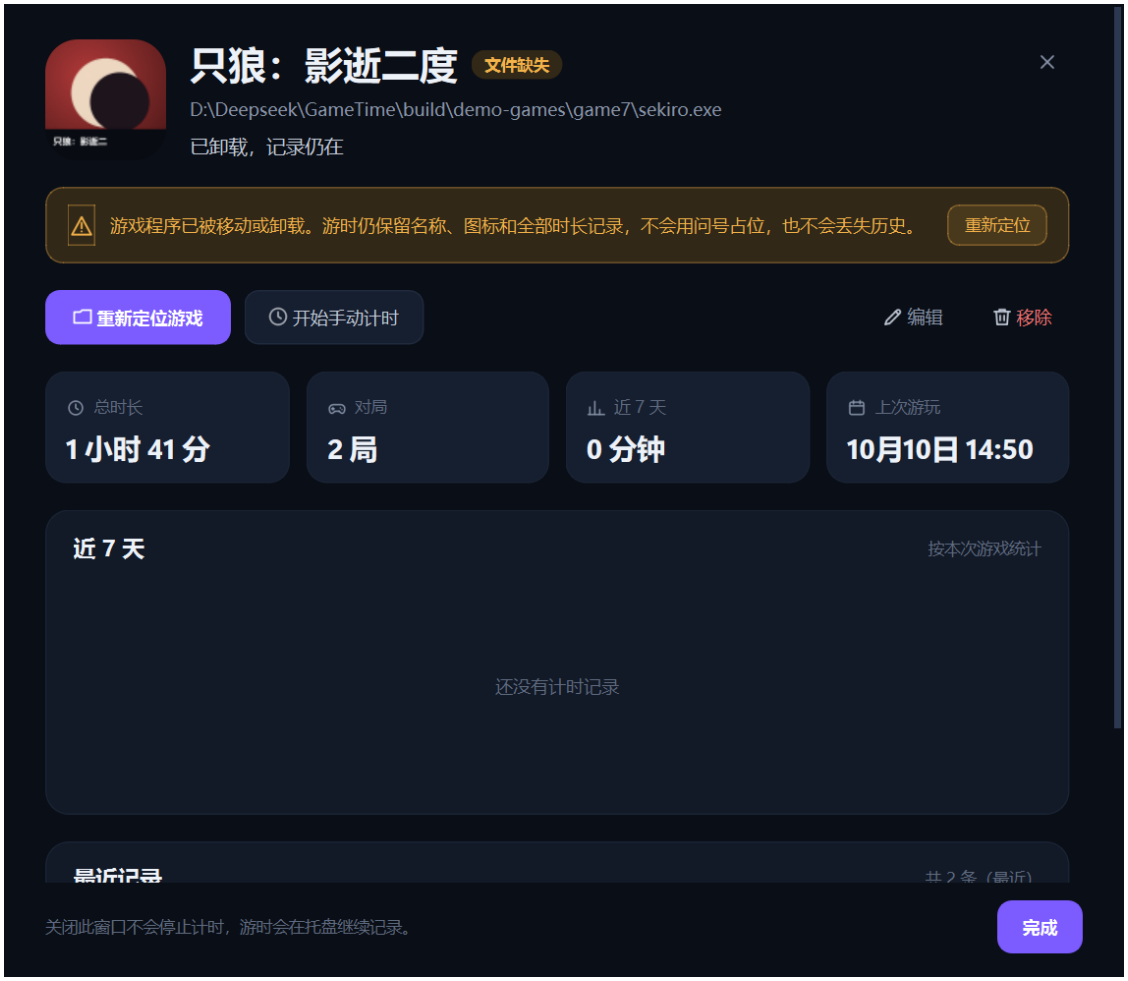


图 8 卸载后的游戏：保留名称、缓存图标与历史，只显示「文件缺失」

03 使用说明

3.1 首次使用

01 解压便携包到任意有写权限的目录，保留 _internal 文件夹。

02 双击 GameTime.exe。

03 点击「添加游戏」手动选择 exe，或点击「扫描游戏」从候选里勾选。

04 正常启动游戏，或从卡片上的播放按钮启动。

05 游戏进程出现后自动计时，退出后自动保存。

扫描添加

点击「扫描游戏」后，后台线程读取 Steam、Epic 与常见 Games 目录，结果会显示名称、路径、来源和“已添加”状态。默认全盘扫描所有本地磁盘，可发现 D:\APP2 等自定义目录；可点击「添加文件夹…」补充目录，勾选候选并确认后才写入游戏库。

数据目录

```
%LOCALAPPDATA%\GameTime\
├─ gametime.db          SQLite 数据库
├─ icons\               永久缓存的游戏图标
├─ logs\gametime.log    运行日志
└─ exports\             默认 CSV 导出目录
```

3.2 计时流程

场景	操作	预期结果
直接启动游戏	启动游戏 exe	2 秒内出现「计时中」，退出后自动结束
从游时启动	点击卡片播放按钮	启动游戏；进程出现后自动计时
启动器拉起其他 exe	详情页手动开始 / 结束	按手动会话记录，不依赖进程识别
游戏由管理员运行	详情页手动计时	避免进程路径不可读导致的自动识别失败
关闭主窗口	点击 X	隐藏到托盘，计时继续
彻底退出	托盘菜单 → 退出游时	结束当前会话并保存，进程退出

3.3 卸载、移动与重新定位

游戏卸载或移动后不需要任何操作。下次刷新会显示「文件缺失」，历史仍在。重装到新位置时，打开详情 →「重新定位」→ 选择新 exe，名称、图标和历史继续沿用。只有详情 →「移除」会删除该游戏的条目和全部历史，操作不可撤销。

3.4 快捷键与数据备份

快捷键	功能
Ctrl + N	添加游戏
Ctrl + Shift + S	扫描游戏
Ctrl + F	进入游戏库并聚焦搜索
Esc	关闭当前游戏详情

备份：退出游时后复制整个 %LOCALAPPDATA%\GameTime 目录即可；迁移时复制到另一台电脑的相同位置。CSV 通过设置页或统计页导出，编码为 UTF-8 BOM，可直接用 Excel 打开。

04 框架与技术选型

4.1 技术栈

层	技术	版本	作用
语言	Python	3.14.3	业务逻辑、UI、打包脚本
GUI	PySide6 / Qt	6.12.0	原生控件、托盘、文件对话框、高 DPI
数据	SQLite	3.x（内置）	游戏、会话、设置持久化，WAL 模式
进程	psutil	7.2.2	枚举进程与 exe 路径
打包	PyInstaller	6.20.0	Windows x64 便携版
测试	pytest	9.1.1	单元、服务、UI、真实进程集成测试

4.2 为什么选择 Python + PySide6

- **原生体验：**Qt Widgets 是原生 C++ 控件封装，托盘、文件选择器、高 DPI 行为与 Windows 一致，不套浏览器。
- **开发效率：**计时、SQLite、图标、进程、扫描、主题、CSV 等核心逻辑用 Python 实现直接、可测试。

- **运行开销**：相比 Electron 省去 Chromium；便携目录约 121 MB，压缩包约 51 MB。
- **系统集成**：QSystemTrayIcon、QProcess、QFileIconProvider、QLocalServer 等能力可直接使用。
- **主题能力**：QSS + QPalette + 自绘控件可完成完整深浅色主题，不需要 Web UI。
- **许可**：PySide6 采用 LGPLv3，本项目动态链接 Qt 共享库，未修改 Qt 源码。

4.3 备选方案对比

方案	未采用原因
Electron + Node	包体和内存开销大，对托盘小工具过重
Tauri + Rust + WebView	需要 WebView2 与 Rust 工具链；UI 仍是 Web，打包复杂度更高
C# / WPF	Windows 原生体验好，但用户优先 Python；计时逻辑与测试迭代成本更高
Tkinter	自带、轻量，但高 DPI、托盘、现代视觉与复杂布局能力不足
Qt / C++	体验与性能合适，但开发成本高，迭代慢

> **结论**
>
> 对“原生 Windows 小工具 + 本地数据 + 托盘常驻”的需求，Python + PySide6 是开发效率、界面质量与运行成本之间的平衡点。若未来需要更极致的启动速度或单文件体积，可评估 C# / WPF 重写，但当前方案已经满足功能与部署要求。

4.4 架构

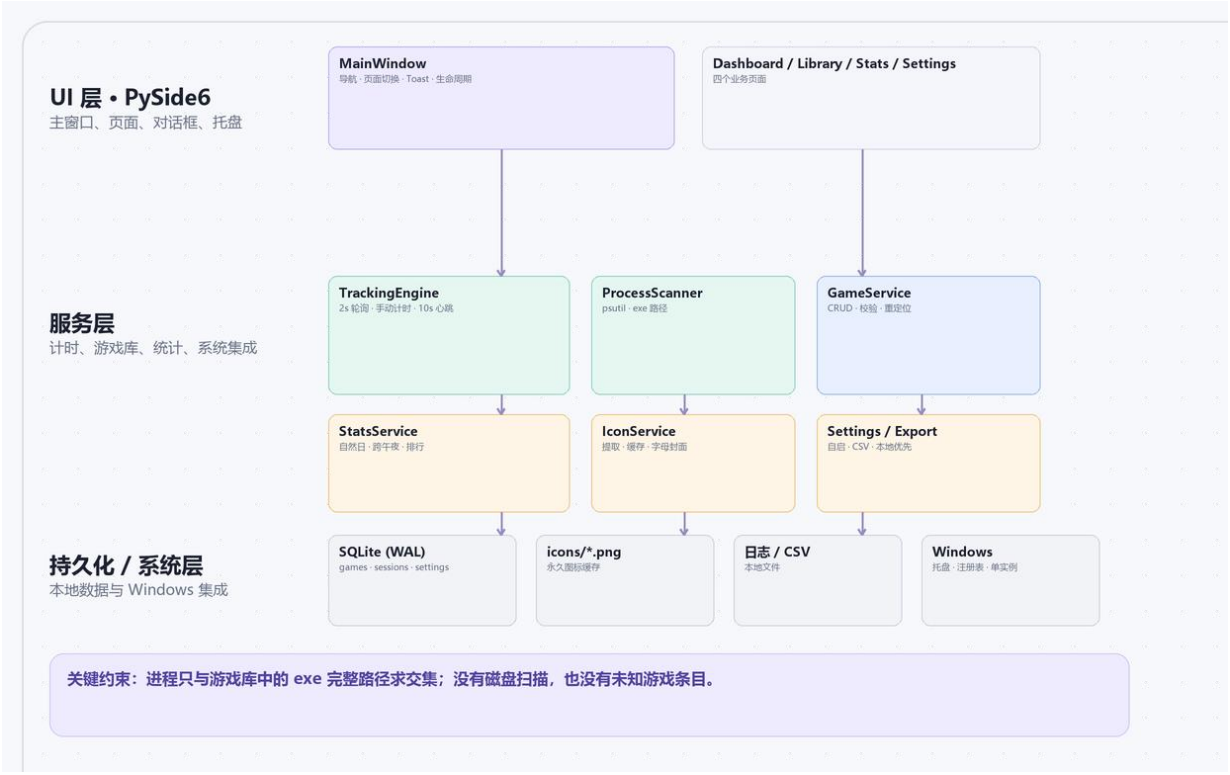


图 9 架构：UI 层、服务层与持久化 / 系统层

图中的服务层为核心计时/存储链路；扫描线程（GameScanner / ScanThread）与主题管理（ThemeManager）分别在 5.2、5.3 展开，二者都不会绕过“用户确认”和“游戏库”边界。

05 关键实现

5.1 只记录添加过的游戏

ProcessScanner 每轮返回系统中所有可读进程的规范化 exe 路径集合。TrackingEngine 取游戏库中的所有游戏，用游戏 exe 路径与进程集合求交集。计时链路没有目录扫描、进程名猜测或商店 API；扫描候选必须经用户确认后才会进入游戏库，因此未知 exe 即使正在运行也不会被计时。

```
games = library.list_games()
running = ProcessScanner().running_paths()   # 所有进程的规范化 exe 路径

for game in games:
    if normalize(game.exe_path) in running:
        start_or_keep_session(game)
```

5.2 自动扫描链路

点击「扫描游戏」后，后台线程读取 Steam、Epic、常见 Games 目录，并默认遍历所有本地固定磁盘。全盘扫描会跳过 Windows、WinSxS、WindowsApps、包缓存与开发缓存，按路径特征、文件大小、目录层级评分，过滤安装器、运行库、平台启动器、微端/网页客户端和已知工具。结果只作为候选返回，必须由用户勾选确认后才调用 GameService.add_game() 写入游戏库。

平台清单通常准确；全盘/目录启发式仍可能把启动器或工具列为候选，因此界面会显示路径与来源，并强制确认。全盘扫描本机约 10–25 秒，可取消勾选做快速扫描；关闭扫描对话框会中断后台线程，未确认的结果不会入库。

5.3 主题管理

ui/theme.py 维护 COLORS_DARK / COLORS_LIGHT 两套调色板和完整 QSS。切换时更新全局 COLORS、Qt QPalette 与样式表，重建主窗口内容并更新原生标题栏明暗。主窗口重建时保留当前页面、计时服务、托盘和快捷键； “跟随系统”每 3 秒读取 AppsUseLightTheme，在无模态窗口时自动应用。

5.4 会话与时间精度

```
idle —进程出现 / 手动开始—> active
active —进程退出 / 手动结束 / 应用退出—> completed

每次轮询：
  duration += min(now - last_tick, 5 秒)
  每 10 秒持久化 duration 与 last_heartbeat
应用异常退出后：
  下次启动以 last_heartbeat 结束遗留 active 会话
```

- **睡眠保护**：正常 2 秒轮询按实际时长累计；电脑睡眠或事件循环卡顿最多只计 5 秒。
- **崩溃恢复**：每 10 秒写心跳，强制结束进程最多损失一个心跳周期，而不是把整段挂起时间算进去。
- **时区**：所有时间戳存 Unix 秒（UTC），显示时转换为本地时间。

5.5 统计与跨午夜

统计按本地自然日聚合。对跨午夜会话，按墙钟区间重叠比例把时长拆分到两天；例如 23:30–00:30 的 1 小时会话会拆成 1800 秒 + 1800 秒。今日、近 7 天、近 14 天与排行都来自同一批会话数据。

5.6 卸载状态如何保证

- **图标**：图标在添加时复制到 icons/，数据库只存相对文件名，读取时校验路径必须在 icons 目录内。
- **提取**：通过 Windows Shell 读取 exe 的真实图标，裁掉透明留白并归一化为 256×256；旧版小图标缓存会在升级后自动修复一次。
- **显示**：UI 不根据 exe 是否存在来决定显示“未知”，而是读取缓存的名称/图标，单独标记文件缺失。
- **兜底**：缓存也丢失时，立即用名称和强调色生成字母封面，仍然不是问号或未知图标。
- **历史**：只有用户确认移除才删除记录；卸载游戏本身不会触发数据库清理。

5.7 单实例与托盘

QLocalServer + QLocalSocket 实现单实例，服务名包含用户与数据目录哈希。第二个实例发送 show 后退出，第一个实例恢复并置前窗口。关闭窗口默认只 hide()，计时不停；如果系统托盘不可用，关闭窗口会直接退出，避免应用“消失且无法找回”。托盘菜单对象由主窗口持有，防止 Python GC 导致菜单失效。

5.8 数据库结构

```
games(id, name, exe_path UNIQUE NOCASE, exe_name, icon_file,
      accent, notes, created_at, updated_at)
sessions(id, game_id → games.id ON DELETE CASCADE,
         started_at, ended_at, duration_seconds, last_heartbeat,
         source CHECK(auto/manual), status CHECK(active/completed))
settings(key PRIMARY KEY, value)
-- 每个游戏最多一个 active 会话（部分唯一索引）
```

06 测试报告

测试在 Windows 11 x64、Python 3.14.3、PySide6 6.12.0 环境完成。自动化覆盖数据层、游戏库、计时引擎、扫描、主题、统计、UI 与真实进程；打包版另行执行 smoke test、单实例、托盘与端到端自动计时。完整明细见 docs/测试报告.md。

6.1 自动化测试

> **测试结果**
>
> python -m pytest -q → 58 passed。未发现阻断使用的缺陷。

测试文件	数量	覆盖内容
test_database.py	9	schema、唯一约束、级联、会话生命周期、崩溃恢复、排序搜索
test_library.py	5	添加/编辑/重定位/移除、路径校验、图标缓存、卸载元数据
test_tracking.py	9	只追踪库内游戏、自动/手动、睡眠上限、心跳、冲突、恢复
test_stats.py	5	今日/7 天/累计、跨午夜拆分、活动会话、空数据
test_export_settings.py	4	CSV 编码字段、设置、自启命令
test_scanner.py	8	Steam/Epic 清单、全盘扫描、注册表命名、自定义目录、去重、过滤
test_theme.py	3	调色板、系统跟随、主窗口主题重建
test_ui.py	14	导航、添加、扫描确认、卸载状态、手动计时、删除、托盘
test_process_integration.py	1	真实 ping.exe 进程识别、计时、退出、入库
合计	58	

关键断言包括：卸载后详情所有文本不含“未知”、模拟 3600 秒睡眠间隔只累计 5 秒、23:30–00:30 会话拆分为 1800+1800 秒、真实进程自动会话 6 秒、浅色主题重启后生效、扫描结果必须确认才入库。

6.2 打包版与 Windows 集成

验证项	结果
PyInstaller 打包 + smoke test	通过，输出 JSON ok=true、theme_loaded=true
演示数据加载	通过，game_count=7、window_visible=true
主题启动	保存浅色主题后重启，theme_effective=light
正常启动	进程 Responding=True，窗口标题正确，托盘可用
单实例	第二实例退出码 0；首实例窗口句柄由 0 变为有效值
扫描	全盘扫描实机发现 8 个候选（含《以闪亮之名》），结果需勾选确认
端到端自动计时	active→completed，自动记录 6 秒，日志包含 started/stopped
真实托盘 / 关闭到托盘 / 通知 API	全部 PASS
HKCU 开机自启	写入、读取、删除、恢复全部 PASS

Windows 集成验证输出：PASS tray_created、tray_open_action、tray_toggle、close_to_tray、tray_notification_api、autostart_registry，最终 ALL_WINDOWS_FEATURE_CHECKS_PASSED。

局限：未在真机商业游戏上测试（使用真实 ping.exe 进程替代），Windows 通知实际可见性取决于系统通知设置；未做 24 小时挂机与全 DPI 穷举。

07 部署与限制

7.1 系统要求与包内容

项目	要求 / 内容
操作系统	Windows 10 1809+ / Windows 11，x64
权限	普通用户；无需管理员，无需 .NET / Java / Node.js / Python
磁盘	约 120 MB（解压后）
包结构	GameTime.exe、_internal\、docs\、LICENSES\、README.md
数据	%LOCALAPPDATA%\GameTime（SQLite + 图标 + 日志）
分发	未签名便携 ZIP；正式分发建议代码签名

7.2 从源码运行与构建

```
# 安装开发依赖
python -m pip install -r requirements-dev.txt

# 源码运行
python -m gametime

# 静态检查 + 测试
python -m ruff check gametime tests scripts
python -m pytest -q

# 构建便携包（自动 smoke test）
powershell -ExecutionPolicy Bypass -File .\scripts\build.ps1
```

7.3 已知限制

限制	说明 / 应对
扫描是启发式	全盘扫描可能列出启动器/工具；默认只保留评分较高的候选，添加前核对
全盘扫描耗时	需遍历所有本地磁盘，本机约 10–25 秒；可取消勾选改用快速扫描
按 exe 完整路径匹配	启动器拉起另一个 exe、管理员权限导致路径不可读时，请用手动计时
跟随系统延迟	每 3 秒检测；读取失败回退深色，模态窗口期间延后应用
非反作弊 / 非平台插件	不注入游戏、不读取内存、不提供 overlay 或帧率
睡眠上限	超过 5 秒的轮询间隔不计入，避免把睡眠算成游玩
异常退出	最多损失一个 10 秒心跳周期
手动会话	正常退出会结束；崩溃后下次启动完成已心跳时长
未签名	SmartScreen 可能提示“未知发布者”，属预期
无云同步	数据只在本机，自行复制数据目录备份
通知	可能被勿扰模式或系统通知设置拦截

7.4 隐私与数据

游时没有网络请求、账号、遥测或广告。数据只写入 %LOCALAPPDATA%\GameTime。扫描只读取本机平台清单与目录，结果仅在本地展示、不上传；日志只记录启动、计时事件与错误堆栈，不记录游戏文件内容。CSV 导出由用户主动触发。卸载软件不会自动删除数据；彻底清除需手动删除该目录。

08 交付与声明

8.1 交付物

路径	说明
dist\GameTime\GameTime.exe	可直接运行的便携版
dist\GameTime-1.1.2-win64-portable.zip	便携压缩包（推荐交付）
docs\游时-说明文档.md / .docx / .pdf	本说明文档
docs\测试报告.md	测试明细与证据
docs\images	真实界面截图与文档配图
gametime\ / tests	完整源码与 58 项测试
GameTime.spec / scripts\build.ps1	打包配置与构建脚本
scripts\verify_windows_features.py	Windows 托盘 / 自启验证脚本
scripts\verify_scan_runtime.py	实机扫描线程验证脚本

8.2 第三方组件

组件	许可	用途
Python	PSF License	运行时
PySide6 / Qt 6	LGPLv3 / 商业许可	GUI、托盘、图标、网络
SQLite	Public Domain	本地数据库
psutil	BSD-3-Clause	进程枚举
PyInstaller	GPLv2 + 启动器例外	打包

游时以动态链接方式使用 Qt / PySide6，未修改 Qt 源码。完整 LGPL/GPL 文本位于包内 LICENSES 目录，第三方组件与源代码链接见 THIRD_PARTY_NOTICES.md。